

Python Sets: A Reference

A set in Python is an **unordered** collection of **unique** items enclosed in curly braces (`{ }`). Sets automatically eliminate duplicate values and do not track position indexes, making them fast for checking membership and performing mathematical operations like unions or intersections.

1. Basic Mechanics & Unique Elements

Tool / Syntax	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>{item1, item2}</code> (Creation)	Defines a set using curly braces. Duplicate values are dropped automatically. Note: To make an empty set, you must use <code>set()</code> , as <code>{ }</code> creates an empty dictionary.	<pre>>>> items = {1, 2, 2, 3, 3} >>> items {1, 2, 3} # Duplicates disappeared >>> empty = set() # Correct empty set >>> type(empty) <class 'set'></pre>
<code>in</code>	Checks for presence. Set membership testing is extremely fast compared to lists or tuples.	<pre>>>> unique_ids = {"ID1", "ID2"} >>> "ID1" in unique_ids True</pre>

2. Adding & Removing Items

Method	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>.add(item)</code>	Adds a new unique item into the set. If the item already exists, the set remains completely unchanged.	<pre>>>> emails = {"a@test.com"} >>> emails.add("b@test.com") >>> emails.add("a@test.com") # Duplicate ignored >>> emails {'a@test.com', 'b@test.com'}</pre>
<code>.remove(val)</code>	Removes a specific value. Throws a KeyError if the value does not exist in the set.	<pre>>>> tags = {"python", "data"} >>> tags.remove("data") >>> tags {'python'}</pre>
<code>.discard(val)</code>	Removes a specific value. Unlike remove, if the value is missing, it passes silently without throwing an error.	<pre>>>> tags = {"python"} >>> tags.discard("java") # Safe to run >>> tags {'python'}</pre>

3. Set Operations (Venn Diagram Math)

Method / Operator	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>.union()</code> or <code> </code>	Union: Combines all unique elements from both sets into a single collection.	<pre>>>> s1 = {1, 2} >>> s2 = {2, 3} >>> s1.union(s2) {1, 2, 3}</pre>
<code>.intersection()</code> or <code>&</code>	Intersection: Finds and isolates only the elements that exist in both sets.	<pre>>>> s1 = {1, 2} >>> s2 = {2, 3} >>> s1.intersection(s2) {2}</pre>
<code>.difference()</code> or <code>-</code>	Difference (Asymmetric): Returns elements uniquely inside the left set but not in the right set. The order matters.	<pre>>>> s1 = {1, 2} >>> s2 = {2, 3} >>> s1.difference(s2) {1} >>> s2.difference(s1) {3}</pre>
<code>.symmetric_difference()</code> or <code>^</code>	Symmetric Difference: Returns elements that are in one of the sets, but not in both (everything except the intersection). Order does not matter.	<pre>>>> s1 = {1, 2} >>> s2 = {2, 3} >>> s1.symmetric_difference(s2) {1, 3}</pre>